SQL JOINs AND SET OPERATIONS

Comprehensive Guide

Dr. Y Subbarayudu | Department CSE | KL University

Practical Execution with Real-World Examples

SECTION 1: CROSS JOIN

A Cartesian product that pairs every row from the first table with every row from the second table.

Screenshot 1: Table Creation & Data Insertion

```
mysql> /* ========================================
    /*>    FILE 04: JOINS AND SET OPERATIONS
    /*>    ========================================
mysql>
mysql> CREATE TABLE class (
    ->       id INT,
    ->       name VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE class_info (
    ->       id INT,
    ->       address VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> INSERT INTO class VALUES
    -> (1,'abhi'),
    -> (2,'adam'),
    -> (3,'alex'),
    -> (4,'anu'),
    -> (5,'ashish');
Query OK, 5 rows affected (0.00 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO class_info VALUES
    -> (1,'DELHI'),
    -> (2,'MUMBAI'),
    -> (3,'CHENNAI'),
    -> (7,'NOIDA'),
    -> (8,'PANIPAT');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Created two tables: class (5 students) and class_info (5 addresses)

Screenshot 2: CROSS JOIN Result

```
mysql> /* CROSS JOIN */
mysql>
mysql> SELECT *
    -> FROM class
    -> CROSS JOIN class_info;
+------+---------+------+-----------+
| id   | name    | id   | address   |
+------+---------+------+-----------+
|    5 | ashish  |    1 | DELHI     |
|    4 | anu     |    1 | DELHI     |
|    3 | alex    |    1 | DELHI     |
|    2 | adam    |    1 | DELHI     |
|    1 | abhi    |    1 | DELHI     |
|    5 | ashish  |    2 | MUMBAI    |
|    4 | anu     |    2 | MUMBAI    |
|    3 | alex    |    2 | MUMBAI    |
|    2 | adam    |    2 | MUMBAI    |
|    1 | abhi    |    2 | MUMBAI    |
|    5 | ashish  |    3 | CHENNAI   |
|    4 | anu     |    3 | CHENNAI   |
|    3 | alex    |    3 | CHENNAI   |
|    2 | adam    |    3 | CHENNAI   |
|    1 | abhi    |    3 | CHENNAI   |
|    5 | ashish  |    7 | NOIDA     |
|    4 | anu     |    7 | NOIDA     |
|    3 | alex    |    7 | NOIDA     |
|    2 | adam    |    7 | NOIDA     |
|    1 | abhi    |    7 | NOIDA     |
|    5 | ashish  |    8 | PANIPAT   |
|    4 | anu     |    8 | PANIPAT   |
|    3 | alex    |    8 | PANIPAT   |
|    2 | adam    |    8 | PANIPAT   |
|    1 | abhi    |    8 | PANIPAT   |
+------+---------+------+-----------+
25 rows in set (0.00 sec)
```

Result: 25 rows (5 students × 5 addresses). Every student paired with every address for survey analysis.

SECTION 2: INNER JOIN & NATURAL JOIN

Retrieves only matching records where the join condition is satisfied.

Screenshot 3: INNER JOIN & NATURAL JOIN

```
mysql> /* INNER JOIN */
mysql>
mysql> SELECT *
    -> FROM class
    -> INNER JOIN class_info
    -> ON class.id = class_info.id;
+------+------+------+-----------+
| id   | name | id   | address   |
+------+------+------+-----------+
|    1 | abhi |    1 | DELHI     |
|    2 | adam |    2 | MUMBAI    |
|    3 | alex |    3 | CHENNAI   |
+------+------+------+-----------+
3 rows in set (0.00 sec)

mysql>
mysql> SELECT class.name,
    ->        class_info.address
    -> FROM class
    -> INNER JOIN class_info
    -> ON class.id = class_info.id;
+------+-----------+
| name | address   |
+------+-----------+
| abhi | DELHI     |
| adam | MUMBAI    |
| alex | CHENNAI   |
+------+-----------+
3 rows in set (0.00 sec)

mysql>
mysql>
mysql> /* NATURAL JOIN */
mysql>
mysql> SELECT *
    -> FROM class
    -> NATURAL JOIN class_info;
+------+------+-----------+
| id   | name | address   |
+------+------+-----------+
|    1 | abhi | DELHI     |
|    2 | adam | MUMBAI    |
|    3 | alex | CHENNAI   |
+------+------+-----------+
3 rows in set (0.00 sec)
```

• INNER JOIN: Returns 3 rows with matching IDs (abhi-DELHI, adam-MUMBAI, alex-CHENNAI)
• NATURAL JOIN: Automatically joins on common column (id) with same result
• Selective columns: Shows only name and address for registered students

SECTION 3: LEFT OUTER JOIN & RIGHT OUTER JOIN

LEFT JOIN keeps all left table records, RIGHT JOIN keeps all right table records.

Screenshot 4: LEFT JOIN & RIGHT JOIN

```
mysql> SELECT *
    -> FROM class
    -> LEFT OUTER JOIN class_info
    -> ON class.id = class_info.id;
+------+--------+------+---------+
| id   | name   | id   | address |
+------+--------+------+---------+
|    1 | abhi   |    1 | DELHI   |
|    2 | adam   |    2 | MUMBAI  |
|    3 | alex   |    3 | CHENNAI |
|    4 | anu    | NULL | NULL    |
|    5 | ashish | NULL | NULL    |
+------+--------+------+---------+
5 rows in set (0.00 sec)

mysql>
mysql> SELECT *
    -> FROM class
    -> LEFT JOIN class_info
    -> ON class.id = class_info.id
    -> WHERE class_info.id IS NULL;
+------+--------+------+---------+
| id   | name   | id   | address |
+------+--------+------+---------+
|    4 | anu    | NULL | NULL    |
|    5 | ashish | NULL | NULL    |
+------+--------+------+---------+
2 rows in set (0.00 sec)

mysql>
mysql>
mysql> /* RIGHT OUTER JOIN */
mysql>
mysql> SELECT *
    -> FROM class
    -> RIGHT OUTER JOIN class_info
    -> ON class.id = class_info.id;
+------+--------+------+---------+
| id   | name   | id   | address |
+------+--------+------+---------+
|    1 | abhi   |    1 | DELHI   |
|    2 | adam   |    2 | MUMBAI  |
|    3 | alex   |    3 | CHENNAI |
| NULL | NULL   |    7 | NOIDA   |
| NULL | NULL   |    8 | PANIPAT |
+------+--------+------+---------+
5 rows in set (0.00 sec)
```

• LEFT JOIN: Returns 5 rows - all students including anu & ashish with NULL addresses
• LEFT JOIN + WHERE: Finds unmatched students (anu, ashish)
• RIGHT JOIN: Returns 5 rows - all addresses including NOIDA & PANIPAT with NULL students

Screenshot 5: RIGHT JOIN Unmatched Records

```
mysql> SELECT *
    -> FROM class
    -> RIGHT JOIN class_info
    -> ON class.id = class_info.id
    -> WHERE class.id IS NULL;
+------+------+------+-----------+
| id   | name | id   | address   |
+------+------+------+-----------+
| NULL | NULL |    7 | NOIDA     |
| NULL | NULL |    8 | PANIPAT   |
+------+------+------+-----------+
2 rows in set (0.00 sec)

mysql>
mysql>
mysql> /* MYSQL EQUIVALENT OF FULL OUTER JOIN */
mysql>
mysql> SELECT class.id AS class_id,
    ->        class.name,
    ->        class_info.id AS info_id,
    ->        class_info.address
    -> FROM class
    -> LEFT JOIN class_info
    -> ON class.id = class_info.id
    ->
    -> UNION
    ->
    -> SELECT class.id AS class_id,
    ->        class.name,
    ->        class_info.id AS info_id,
    ->        class_info.address
    -> FROM class
    -> RIGHT JOIN class_info
    -> ON class.id = class_info.id;
+----------+---------+---------+-----------+
| class_id | name    | info_id | address   |
+----------+---------+---------+-----------+
|        1 | abhi    |       1 | DELHI     |
|        2 | adam    |       2 | MUMBAI    |
|        3 | alex    |       3 | CHENNAI   |
|        4 | anu     |    NULL | NULL      |
|        5 | ashish  |    NULL | NULL      |
|     NULL | NULL    |       7 | NOIDA     |
|     NULL | NULL    |       8 | PANIPAT   |
+----------+---------+---------+-----------+
7 rows in set (0.00 sec)
```

• RIGHT JOIN + WHERE: Finds unmatched addresses (NOIDA, PANIPAT) not linked to any student

SECTION 4: FULL OUTER JOIN

Returns all records from both tables - matched and unmatched.

Screenshot 6: FULL OUTER JOIN (MySQL Equivalent)



```
mysql> SELECT class.id AS class_id,
    ->        class.name,
    ->        class_info.id AS info_id,
    ->        class_info.address
    -> FROM class
    -> LEFT JOIN class_info
    -> ON class.id = class_info.id
    -> WHERE class_info.id IS NULL
    ->
    -> UNION ALL
    ->
    -> SELECT class.id AS class_id,
    ->        class.name,
    ->        class_info.id AS info_id,
    ->        class_info.address
    -> FROM class
    -> RIGHT JOIN class_info
    -> ON class.id = class_info.id
    -> WHERE class.id IS NULL;
+------------+----------+-----------+-----------+
| class_id   | name     | info_id   | address   |
+------------+----------+-----------+-----------+
|          4 | anu      |      NULL | NULL      |
|          5 | ashish   |      NULL | NULL      |
|       NULL | NULL     |         7 | NOIDA     |
|       NULL | NULL     |         8 | PANIPAT   |
+------------+----------+-----------+-----------+
4 rows in set (0.00 sec)

mysql>
mysql>
mysql> /* SET-OPERATION TABLES */
mysql>
mysql> CREATE TABLE first_table (
    ->       id INT,
    ->       name VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> CREATE TABLE second_table (
    ->       id INT,
    ->       name VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.03 sec)
```

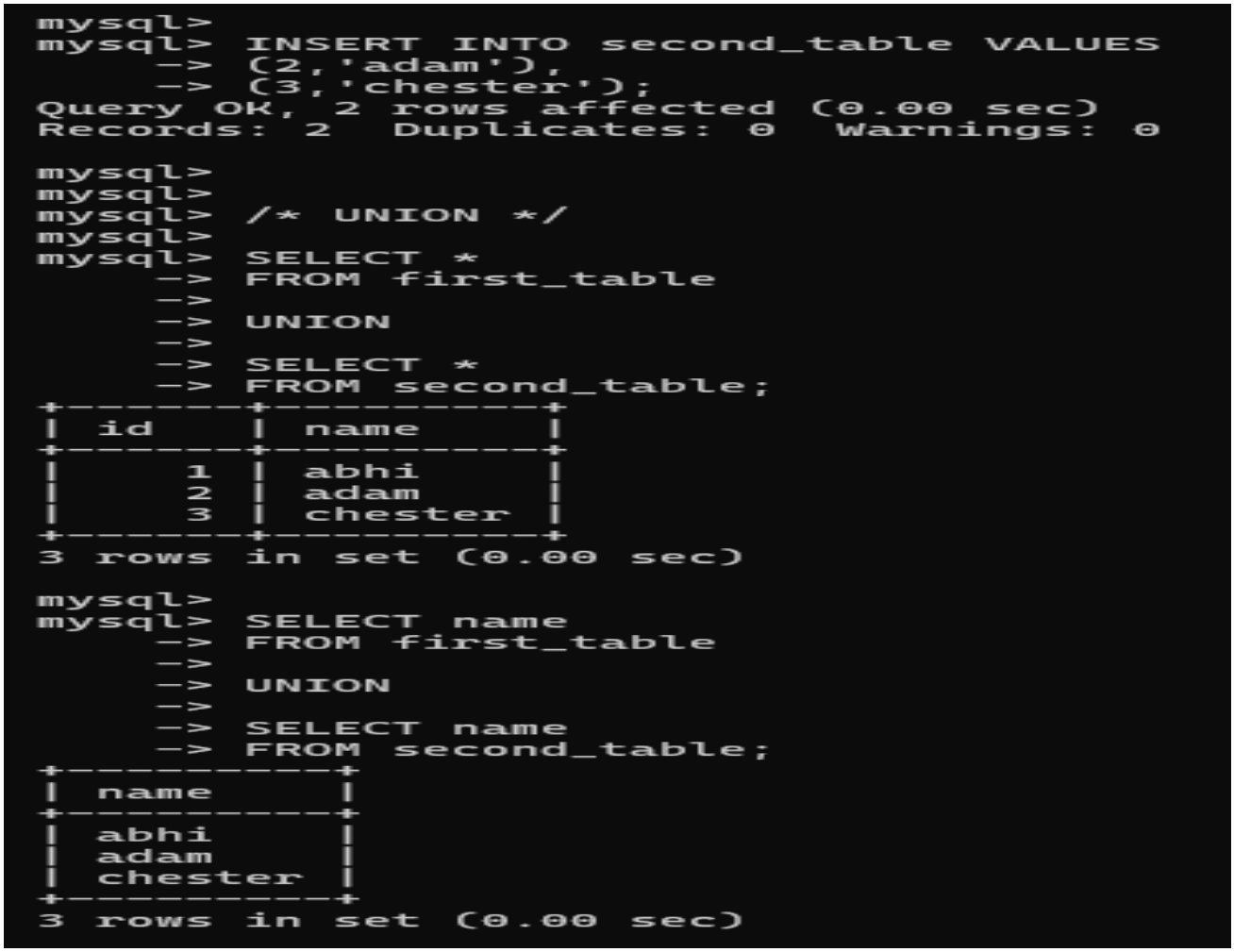
FULL OUTER JOIN achieved through LEFT JOIN UNION RIGHT JOIN:

• Returns 7 rows: All matched records + unmatched students + unmatched addresses

• With WHERE clause: Returns only unmatched records (4 rows)

SECTION 5: SET OPERATIONS (UNION, INTERSECT, MINUS)

Screenshot 7: UNION & UNION ALL



```
mysql>
mysql> INSERT INTO second_table VALUES
    -> (2,'adam'),
    -> (3,'chester');
Query OK, 2 rows affected (0.00 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql>
mysql> /* UNION */
mysql>
mysql> SELECT *
    -> FROM first_table
    ->
    -> UNION
    ->
    -> SELECT *
    -> FROM second_table;
+------+----------+
| id   | name     |
+------+----------+
|    1 | abhi     |
|    2 | adam     |
|    3 | chester  |
+------+----------+
3 rows in set (0.00 sec)

mysql>
mysql> SELECT name
    -> FROM first_table
    ->
    -> UNION
    ->
    -> SELECT name
    -> FROM second_table;
+----------+
| name     |
+----------+
| abhi     |
| adam     |
| chester  |
+----------+
3 rows in set (0.00 sec)
```

• UNION: Combines records from two tables, removes duplicates → 3 rows (abhi, adam, chester)
• UNION (names only): Produces unique student names across both tables

Screenshot 8: UNION ALL & Record Counting



• UNION ALL: Includes duplicates → 4 rows (adam appears twice)
• COUNT with subquery: Counts total records from both tables including duplicates

Screenshot 9: INTERSECT & MINUS Operations

```
mysql> SELECT name
    -> FROM first_table
    ->
    -> INTERSECT
    ->
    -> SELECT name
    -> FROM second_table;
+-------+
| name  |
+-------+
| adam  |
+-------+
1 row in set (0.00 sec)

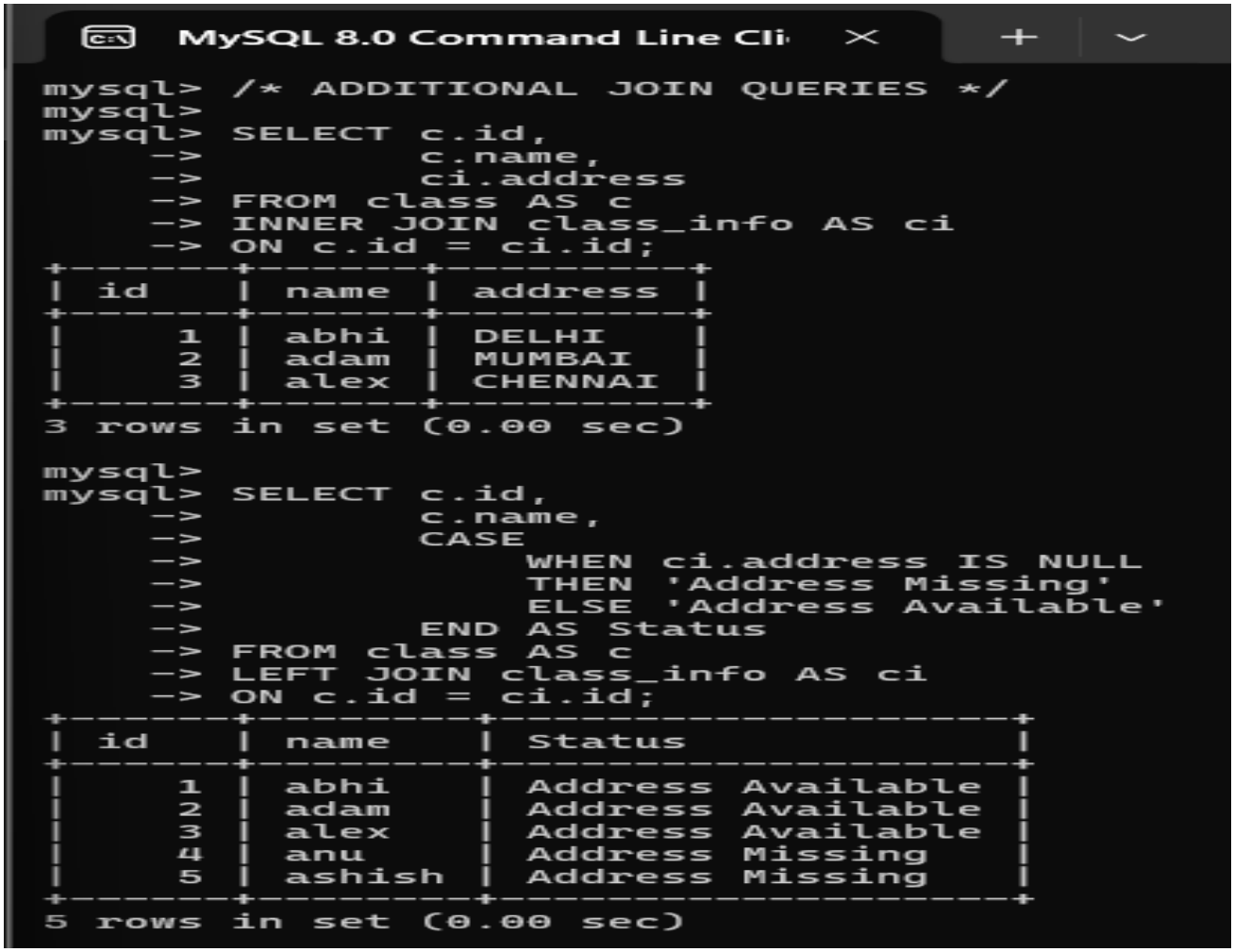
mysql>
mysql>
mysql> /* MYSQL EQUIVALENT OF MINUS */
mysql>
mysql> SELECT first_table.*
    -> FROM first_table
    -> LEFT JOIN second_table
    -> ON first_table.id = second_table.id
    -> AND first_table.name = second_table.name
    -> WHERE second_table.id IS NULL;
+------+-------+
| id   | name  |
+------+-------+
|    1 | abhi  |
+------+-------+
1 row in set (0.00 sec)

mysql>
mysql> SELECT first_table.name
    -> FROM first_table
    -> LEFT JOIN second_table
    -> ON first_table.name = second_table.name
    -> WHERE second_table.name IS NULL;
+-------+
| name  |
+-------+
| abhi  |
+-------+
1 row in set (0.00 sec)
```

• INTERSECT: Finds common records in both tables → Only adam (present in both)
• MINUS (MySQL equivalent): Uses LEFT JOIN to find records only in first table → abhi

SECTION 6: Advanced JOIN Queries

Screenshot 10: INNER JOIN with Aliases & CASE Statement



Advanced Query Patterns:

• Table Aliases: Using AS c and AS ci for cleaner syntax
• INNER JOIN: Matches students with their addresses based on ID
• CASE Statement: Conditional logic to show "Address Available" or "Address Missing"
• LEFT JOIN + CASE: Comprehensive report showing status for all students
- Output: 5 students with their address availability status

COMPREHENSIVE JOIN & SET OPERATION REFERENCE

Operation	Description	Result / Use Case
CROSS JOIN	Cartesian product of all rows	25 rows (5×5 combination for survey analysis)
INNER JOIN	Only matching records	3 rows with confirmed address records
NATURAL JOIN	Auto-join on common columns	3 rows with implicit ID matching
LEFT JOIN	All left table + matching right	5 rows including students without addresses
RIGHT JOIN	All right table + matching left	5 rows including orphaned addresses
FULL OUTER	All records matched and unmatched	7 rows with all data combinations
UNION	Combine tables, remove duplicates	3 unique records (abhi, adam, chester)
UNION ALL	Combine tables, keep duplicates	4 records (adam counted twice)
INTERSECT	Common records in both tables	1 record (adam in both branches)
MINUS	Records only in first table	1 record (abhi only in first_table)

Key Takeaways

✓ CROSS JOIN: Used for combinations (surveys, pairings)
✓ INNER JOIN: Most common - only matched data
✓ LEFT/RIGHT JOIN: Preserve unmatched records from one side
✓ FULL OUTER: Complete data from both tables
✓ UNION: Combines datasets, removes duplicates for clean master lists
✓ INTERSECT: Finds common data across tables
✓ MINUS: Identifies unique records in one table
✓ CASE in JOINs: Adds conditional logic for reporting